

CS219 Project 3 Report

12312109 李泓霖

Introduction

The program only focus on the 24-bits uncompressed BMPs.

Operating Instructions

Input Files

For the input of the images, I use `-i` to represent `input` instruction.

Output Files

For the output of the images, I use `-o` to represent `output` instruction.

Operation Files

For the operation of the instruction, I use `-p` to present `operation` instruction.

Brightness Adjustment

The command line to adjust the brightness of the image is down below.

```
./main -i input.bmp -o output.bmp -p add 50
```

Blending

The command line to blend two bmp images together is down below

```
./main -i input1.bmp -i input2.bmp -o output.bmp -p average
```

Gray-Scale

In this function, the image has been input will have the same `BGR` parameter thus the image output will transform into `Gray-Scale` mode.

```
./main -i input.bmp -o output.bmp -p grayscale
```

Code Explanation

Data Structure Part

In my completed code, struct `BMPFileHeader` stores the data `signature`, `file_size`, `reserved1`, `reserved2` and `data_offset`. Struct `BMPInfoHeader` in my completed code stores more data including `header_size`, `width`, `height`, `planes`, `bits_per_pixel`, `compression`, `image_size`, `x_pixels_per_m`, `y_pixels_per_m`, `colors_used` and `important_colors`. And I created another struct `BMPImage` which stores `file_header`, `info_header`, `pixel_data` and `row_size`, and struct `config` which stores data `output_file`, `input_files`, `input_count`, `operation` and `value`.

Follows are four struct I created in the completed code.

```
typedef struct {
    char signature[2];
    int file_size;
    short reserved1;
    short reserved2;
    int data_offset;
} BmpFileHeader;

typedef struct {
    int header_size;
    int width;
    int height;
    short planes;
    short bits_per_pixel;
    int compression;
    int image_size;
    int x_pixels_per_m;
    int y_pixels_per_m;
    int colors_used;
    int important_colors;
} BmpInfoHeader;

typedef struct {
    BmpFileHeader file_header;
    BmpInfoHeader info_header;
    unsigned char* pixel_data;
    int row_size;
} BmpImage;

#pragma pack(pop)

#define CHECK_NULL(ptr, msg) \
do { \
    if ((ptr) == NULL) { \
        fprintf(stderr, "Error: %s\n", (msg)); \
        exit(EXIT_FAILURE); \
    } \
} while (0)

typedef struct {
    char* output_file;
    char* input_files[2];
    int input_count;
    char* operation;
```

```
    int value;  
} Config;
```

Core Function

IO Function

This part of code's duty is to read the bmp file in, check whether the image is in the right format and output the bmp images after the process of the program. There exist several error messages:

Failed to open file: Means that the file path might be wrong.

Error reading file header: Means that the path to the file is right but the header of the file is wrong or corrupted.

Invalid BMP File: Means that the format of the bmp image didn't fit with the program because of the lack of the **BM** signature.

Error reading info header: Means the info header is corrupted.

Unsupported BMP format: Means that the format of the bmp image is wrong because of the image is not 24 color depth.

Pixel data allocation failed: Means that the image is too large to be process in this program.

Error reading pixel data: Means that the image is not a complete image.

Failed to create output file: Means that the output image has problems.

```
BmpImage* read_bmp(const char* filename) {  
    FILE* file = fopen(filename, "rb");  
    CHECK_NULL(file, "Failed to open file");  
  
    BmpImage* image = malloc(sizeof(BmpImage));  
    CHECK_NULL(image, "Memory allocation failed");  
  
    if (fread(&image->file_header, sizeof(BmpFileHeader), 1, file) != 1) {  
        fprintf(stderr, "Error reading file header\n");  
        exit(EXIT_FAILURE);  
    }  
  
    if (image->file_header.signature[0] != 'B' ||  
        image->file_header.signature[1] != 'M') {  
        fprintf(stderr, "Invalid BMP file\n");  
        exit(EXIT_FAILURE);  
    }  
  
    if (fread(&image->info_header, sizeof(BmpInfoHeader), 1, file) != 1) {  
        fprintf(stderr, "Error reading info header\n");  
        exit(EXIT_FAILURE);  
    }  
}
```

```

if (image->info_header.bits_per_pixel != 24 ||
    image->info_header.compression != 0) {
    fprintf(stderr, "Unsupported BMP format\n");
    exit(EXIT_FAILURE);
}

image->row_size = ((image->info_header.width * 3 + 3) / 4) * 4;
image->pixel_data = malloc(image->row_size * image->info_header.height);
CHECK_NULL(image->pixel_data, "Pixel data allocation failed");

fseek(file, image->file_header.data_offset, SEEK_SET);
if (fread(image->pixel_data, 1, image->row_size * image->info_header.height,
    file) != image->row_size * image->info_header.height) {
    fprintf(stderr, "Error reading pixel data\n");
    exit(EXIT_FAILURE);
}

fclose(file);
return image;
}

void write_bmp(const char* filename, BmpImage* image) {
    FILE* file = fopen(filename, "wb");
    CHECK_NULL(file, "Failed to create output file");

    fwrite(&image->file_header, sizeof(BmpFileHeader), 1, file);
    fwrite(&image->info_header, sizeof(BmpInfoHeader), 1, file);

    fseek(file, image->file_header.data_offset, SEEK_SET);
    fwrite(image->pixel_data, 1, image->row_size * image->info_header.height, file);

    fclose(file);
}

```

Image Process

Brightness Adjustment

In this function, I just adjust the each pixels **BGR** value to adjust the brightness of the bmp image.

There is a truncation mechanism. This mechanism is to make sure that all the **BGR** value is between 0 to 255 so that the image generated will be no errors.

Blending Images

In this function, the **BGR** value of each pixel just simply added together and divided by 2 to generate the new image.

Things to be mentioned is that when two images in different size were input, the program will end and output an error message `Image dimensions mismatch`. That means the two picture you wanted to blend must in same size.

Gray-Scale

This idea comes from another course I have attended in this semester, which is "Computer Vision".

To trans the colorful bmp image into gray-scale, I use a formula to calculate the `BGR` parameters.

$$\text{gray} = (R * 77 + G * 150 + B * 29 + 128) / 256$$

After this operation, the colorful image can be changed into gray-scale mode.

```
void adjust_brightness(BmpImage* image, int delta) {
    unsigned char* ptr = image->pixel_data;
    int total_bytes = image->row_size * image->info_header.height;

    for (int i = 0; i < total_bytes; ++i) {
        int val = (int)ptr[i] + delta;
        ptr[i] = (unsigned char)fmax(0, fmin(255, val));
    }
}

void average_blend(BmpImage* img1, BmpImage* img2, BmpImage* result) {
    if (img1->info_header.width != img2->info_header.width ||
        img1->info_header.height != img2->info_header.height) {
        fprintf(stderr, "Image dimensions mismatch\n");
        exit(EXIT_FAILURE);
    }

    unsigned char* p1 = img1->pixel_data;
    unsigned char* p2 = img2->pixel_data;
    unsigned char* pres = result->pixel_data;
    int total_bytes = img1->row_size * img1->info_header.height;

    for (int i = 0; i < total_bytes; ++i) {
        pres[i] = (p1[i] + p2[i]) / 2;
    }
}

void convert_to_grayscale(BmpImage* image) {
    unsigned char* ptr = image->pixel_data;
    int width = image->info_header.width;
    int height = image->info_header.height;

    for (int y = 0; y < height; ++y) {
        for (int x = 0; x < width; ++x) {
            int offset = y * image->row_size + x * 3;
```

```

    unsigned char b = ptr[offset];
    unsigned char g = ptr[offset + 1];
    unsigned char r = ptr[offset + 2];

    unsigned char gray = (unsigned char)((r * 77 + g * 150 + b * 29 +
128) / 256);

    ptr[offset] = gray;
    ptr[offset + 1] = gray;
    ptr[offset + 2] = gray;
}
}
}

```

Memory Management

This function is to free the memory the image uses to increase the speed of the program and make sure the memory won't leak.

In the function, the `pixel_data` will be free first and after that the struct `image` will be free. In this kind of mode, the free process will not cause big problems.

```

void free_bmp(BmpImage* image) {
    if (image) {
        free(image->pixel_data);
        free(image);
    }
}

```

Command Line Process

In this part, the input command line is processed and stored into its struct `Config`. The way how the command line should be like is listed in the `Operating Instruction` part.

Different with the example in the assignment document, I didn't use `-op` to represent `operation`. There is a reason why I didn't use `-op` but `-p`.

The reason is down below in the `parse_arguments` part. In this part, I directly use command line parameter. But when I trying to use `getpot` method to deal with the command line, I found it is very hard to deal with multiple character.

And after I did a simple research about command line, I found that when I use multiple character commands, we need to use `--command` to represent multiple character commands. To make the commands formats more unified, I use `-p` to represent `operation` command.

There exist some error messages:

Missing required arguments: Means that the command line had been input exist wrong places.

Average operation requires two input files: This error message is especially for `average_blend` function. When the input images is not 2, this error message will come out.

```
void parse_arguments(int argc, char* argv[], Config* config) {
    int opt;
    config->input_count = 0;
    config->operation = NULL;
    config->value = 0;

    while ((opt = getopt(argc, argv, "i:o:p:")) != -1) {
        switch (opt) {
            case 'i':
                if (config->input_count < 2) {
                    config->input_files[config->input_count++] = optarg;
                }
                break;
            case 'o':
                config->output_file = optarg;
                break;
            case 'p':
                if (optarg) {
                    config->operation = optarg;
                    if (strcmp(optarg, "add") == 0 && optind < argc) {
                        config->value = atoi(argv[optind++]);
                    }
                }
                break;
            default:
                fprintf(stderr, "Usage: %s -i input1.bmp [-i input2.bmp] -o
output.bmp "
                        "-p [add|average|grayscale] [value]\n", argv[0]);
                exit(EXIT_FAILURE);
        }
    }

    if (config->input_count == 0 || !config->output_file || !config->
operation) {
        fprintf(stderr, "Missing required arguments\n");
        exit(EXIT_FAILURE);
    }

    if (strcmp(config->operation, "average") == 0 && config->input_count !=
2) {
        fprintf(stderr, "Average operation requires two input files\n");
        exit(EXIT_FAILURE);
    }
}
```

Main Program (Top Module)

This part is the main program, the top module of the whole program. The functions above are called in this part.

```
int main(int argc, char* argv[]) {
    Config config;
    parse_arguments(argc, argv, &config);

    BmpImage* img1 = read_bmp(config.input_files[0]);
    BmpImage* result = NULL;

    if (strcmp(config.operation, "add") == 0) {
        adjust_brightness(img1, config.value);
        result = img1;
    } else if (strcmp(config.operation, "average") == 0) {
        if (config.input_count != 2) {
            fprintf(stderr, "Average operation requires two input files\n");
            exit(EXIT_FAILURE);
        }
        BmpImage* img2 = read_bmp(config.input_files[1]);
        result = malloc(sizeof(BmpImage));
        memcpy(result, img1, sizeof(BmpImage));
        result->pixel_data = malloc(img1->row_size * img1-
>info_header.height);
        memcpy(result->pixel_data, img1->pixel_data,
            img1->row_size * img1->info_header.height);
        average_blend(img1, img2, result);
        free_bmp(img2);
    } else if (strcmp(config.operation, "grayscale") == 0) {
        convert_to_grayscale(img1);
        result = img1;
    } else {
        fprintf(stderr, "Unknown operation: %s\n", config.operation);
        exit(EXIT_FAILURE);
    }

    write_bmp(config.output_file, result);

    free_bmp(result);
    if (result != img1) {
        free_bmp(img1);
    }

    return 0;
}
```

Testcases

Generation of Testcases

To perfectly fit with the format requirement, I used Deepseek to generate a program which can generate proper format that the assignment needs. The program will generate a $256 * 256$ BMP image which has

16 different colors which format is 24-bits and uncompressed. The code will automatically put the generated image into the folder **data** who is in the same direction with the generation code. The code to generate images was put in the **AI Usage** section.

Brightness Adjustment

Color Test

This function can adjust the brightness of the image. Two pictures below show the image itself and after adjust its brightness by **+120**.



Image1: Before Brightness Adjustment



Image2: After Brightness Adjustment

Edge Test

This testcase is to test whether the function can process edge situations. In this testcase I adjust a pure black image and I adjust the brightness by **+200**.

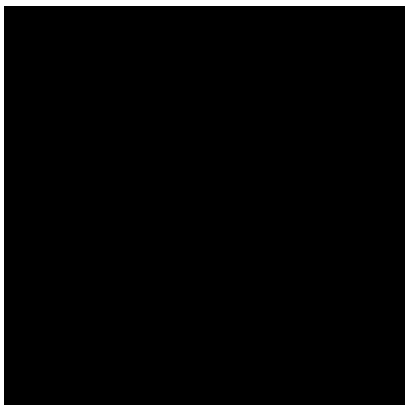


Image3: Before Brightness Adjustment

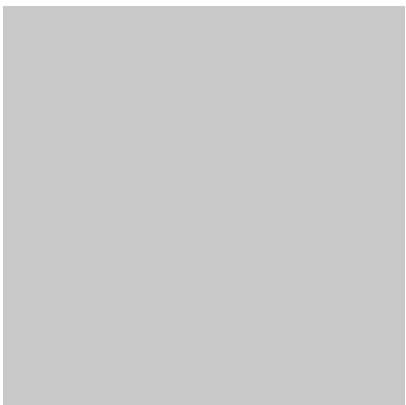


Image4: After Brightness Adjustment

Blending

This function can blending two images by averaging their pixel values.



Image5: Blending
Image1

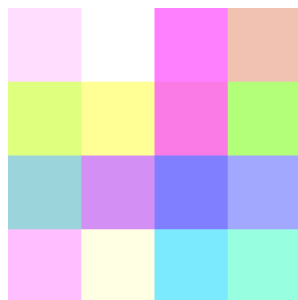


Image6: Blending
Image2



Image7: After Blending

Gray-Scale

This function can turn colorful images into gray-scale.

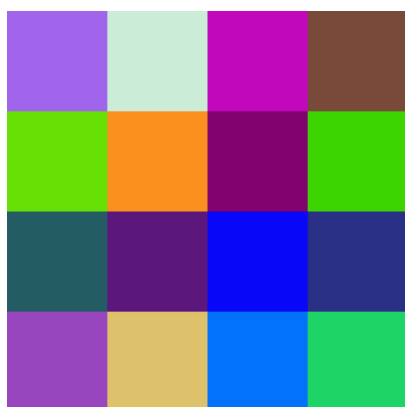


Image8: Before Gray-Scale

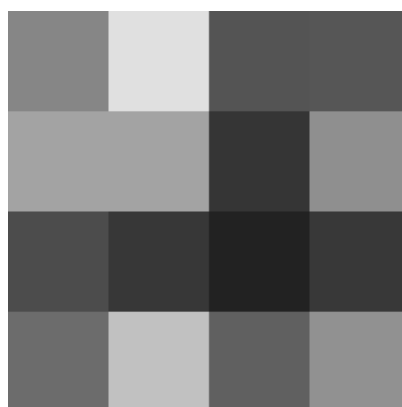


Image9: After Gray-Scale

AI Usage

Generation of the BMP Generation Code

The code following are the code I used to generate the BMP images.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <sys/stat.h>

// 增强版目录创建函数
void create_data_dir() {
    const char* dir_path = "data";

    // 检查目录是否已存在
    struct stat st;
    if (stat(dir_path, &st) == 0) {
        if (S_ISDIR(st.st_mode)) {
            return; // 目录已存在
        } else {

```

```
        fprintf(stderr, "Error: 'data' exists but is not a
directory\n");
        exit(EXIT_FAILURE);
    }
}

// 创建目录 (兼容Windows和Linux)
#ifdef _WIN32
int ret = _mkdir(dir_path);
#else
int ret = mkdir(dir_path, 0777); // 设置权限rwxrwxrwx
#endif

if (ret != 0) {
    perror("Failed to create data directory");
    exit(EXIT_FAILURE);
}
}

#pragma pack(push, 1)
typedef struct {
    char signature[2];
    int file_size;
    short reserved1;
    short reserved2;
    int data_offset;
} BMPFileHeader;

typedef struct {
    int header_size;
    int width;
    int height;
    short planes;
    short bits_per_pixel;
    int compression;
    int image_size;
    int x_pixels_per_m;
    int y_pixels_per_m;
    int colors_used;
    int important_colors;
} BMPInfoHeader;
#pragma pack(pop)

void create_dir(const char* path) {
    #ifdef _WIN32
        _mkdir(path);
    #else
        mkdir(path, 0777);
    #endif
}

int main() {
    create_dir("data"); // 创建目录
```

```
srand(time(NULL));

const int width = 256;
const int height = 256;
const int block_size = 64; // 256/4=64
const int bytes_per_pixel = 3;

// 计算行大小 (4字节对齐)
int row_size = (width * bytes_per_pixel + 3) & ~3;

// 初始化文件头
BMPFileHeader file_header = {
    .signature = {'B', 'M'},
    .file_size = sizeof(BMPFileHeader) + sizeof(BMPInfoHeader) +
row_size * height,
    .reserved1 = 0,
    .reserved2 = 0,
    .data_offset = sizeof(BMPFileHeader) + sizeof(BMPInfoHeader)
};

// 初始化信息头
BMPInfoHeader info_header = {
    .header_size = sizeof(BMPInfoHeader),
    .width = width,
    .height = height,
    .planes = 1,
    .bits_per_pixel = 24,
    .compression = 0,
    .image_size = row_size * height,
    .x_pixels_per_m = 2835,
    .y_pixels_per_m = 2835,
    .colors_used = 0,
    .important_colors = 0
};

// 生成16种随机颜色 (BGR格式)
unsigned char colors[16][3];
for (int i = 0; i < 16; i++) {
    colors[i][0] = rand() % 256; // 蓝
    colors[i][1] = rand() % 256; // 绿
    colors[i][2] = rand() % 256; // 红
}

// 创建像素数组 (从下到上存储)
unsigned char pixels[height][row_size];

// Fill up the 4*4 color blocks
for (int block_y = 0; block_y < 4; block_y++) {
    for (int block_x = 0; block_x < 4; block_x++) {
        // Current color
        int color_index = block_y * 4 + block_x;

        // Calculation of the range of the color block
        int start_x = block_x * block_size;
```

```

        int end_x = (block_x + 1) * block_size;
        int start_y = block_y * block_size;
        int end_y = (block_y + 1) * block_size;

        // Fill up the color block
        for (int y = start_y; y < end_y; y++) {
            for (int x = start_x; x < end_x; x++) {
                int pos = x * bytes_per_pixel;
                pixels[y][pos] = colors[color_index][0]; // 蓝
                pixels[y][pos + 1] = colors[color_index][1]; // 绿
                pixels[y][pos + 2] = colors[color_index][2]; // 红
            }
        }
    }
}

// Output the BMP file.
FILE* file = fopen("data/color_blocks.bmp", "wb");
if (!file) {
    perror("Error creating file");
    return 1;
}

fwrite(&file_header, sizeof(BMPFileHeader), 1, file);
fwrite(&info_header, sizeof(BMPInfoHeader), 1, file);

// Write the bmps from bottom to the top.
for (int y = height-1; y >= 0; y--) {
    fwrite(pixels[y], 1, row_size, file);
}

fclose(file);
printf("生成成功: data/color_blocks.bmp\n");
return 0;
}

```

Generation of Framework

This part shows how I use AI to help me doing my work and which parts are generated by AI.

I used Deepseek to generate the code framework for me. The instruction I gave to AI is "根据这个任务文档帮我写一个基本的C语言的代码框架"

Data Structure Part

This part shows the data structure part that AI generated for me.

```

#pragma pack(push, 1)
/* BMP文件头结构 */
typedef struct {
    char signature[2]; // 文件标识"BM"

```

```

    int file_size;           // 文件总大小
    short reserved1;        // 保留字段
    short reserved2;
    int data_offset;        // 像素数据偏移量
} BMPFileHeader;

/* BMP信息头结构 */
typedef struct {
    int header_size;        // 信息头大小(40)
    int width;              // 图像宽度
    int height;             // 图像高度
    short planes;           // 颜色平面数(1)
    short bits_per_pixel;   // 每像素位数(24)
    int compression;        // 压缩格式(0)
    // ...其他字段...
} BMPInfoHeader;

/* 完整图像结构 */
typedef struct {
    BMPFileHeader file_header;
    BMPInfoHeader info_header;
    unsigned char* pixel_data; // 像素数组
    int row_size;              // 含填充的实际行大小
} BMPImage;
#pragma pack(pop)

```

Core Function

IO Function

When I input the bmp images the `read_bmp` function will not only read the bmp image but while the function read the image, it will also verify the header identifier and the format of the BMP.

```

// 读取BMP文件
BMPImage* read_bmp(const char* filename) {
    // 文件头校验 -> 信息头读取 -> 像素数据加载
    // 包含严格的格式验证 (24位色深、无压缩)
}

// 写入BMP文件
void write_bmp(const char* filename, BMPImage* image) {
    // 按标准格式写入头数据+像素数据
}

```

Image Process

When AI first generated this code to me, there only has two functions. One is `adjust_brightness` and another one is `average_blend`. I add the `gray_scale` function after by myself.

```
// 亮度调整（全像素线性增减）
void adjust_brightness(BMPImage* image, int delta) {
    // 遍历所有像素通道值进行加减操作
}

// 双图混合（平均值算法）
void average_blend(BMPImage* img1, BMPImage* img2, BMPImage* result) {
    // 尺寸校验 -> 逐像素通道取平均
}
```

Memory Management

To make sure the performance is fine and the memory won't leak, AI recommend me to write a function `free_bmp` to free the memory after the process.

```
// 安全释放图像内存
void free_bmp(BMPImage* image) {
    // 先释放像素数据，再释放结构体
}
```

Command Line Process

Struct of Command Line

In this section the command line will be stored into different arrays.

```
typedef struct {
    char* output_file;        // 输出路径
    char* input_files[2];     // 输入文件数组
    int input_count;          // 输入文件数
    char* operation;          // 操作类型
    int value;                 // 操作参数
} Config;
```

Parameter Analysis

At first AI recommended me to use `getopt`, but consider the command example in the assignment document uses multiple word to represent `operation`, I simply changed the code into `getopt_long`. But soon I find when turning to multiple word commands, the input format will be not unified, so in the final version of the code I changed the method into `getopt` again and uses `-p` to represent `operation`.

```
void parse_arguments(int argc, char* argv[], Config* config) {
    // 使用getopt解析参数：
    // -i 指定输入文件（最多2个）
}
```

```
// -o 指定输出文件
// -p 指定操作类型 (add/average/grayscale)

// 参数校验:
// 1. 必须参数检查
// 2. 操作类型与输入文件数量匹配
}
```

Main Program (Top Module)

```
int main(int argc, char* argv[]) {
    // 1. 参数解析
    Config config;
    parse_arguments(argc, argv, &config);

    // 2. 读取基础图像
    BMPImage* base_img = read_bmp(config.input_files[0]);

    // 3. 操作分发
    BMPImage* result = NULL;
    if (操作类型 == "add") {
        adjust_brightness(base_img, config.value);
        result = base_img;
    } else if (操作类型 == "average") {
        // 读取第二张图并混合
    }

    // 4. 结果输出
    write_bmp(config.output_file, result);

    // 5. 资源清理
    free_bmp(result);
    if (result != base_img) free_bmp(base_img);
}
```

Format

I used AI to fix my codes format into Google C++ Guide format.